

OSSP PRACTICAL 1-6

PRACTICAL-1-

Q) Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that

1. Accept a Linux command as input.
2. Create a child process using `fork()`.
3. Execute the command in the child process using an appropriate `exec()` system call.
4. Allow the parent process to wait for the child using `wait ()`.
5. Display the Process ID (PID) of both parent and child processes.

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>

int main()
{
    char command[100];

    printf("Enter a linux command: ");
    scanf("%s", command);

    int pid;
    pid = fork();

    if(pid == 0)
    {
        printf("Child PID: %d\n", getpid());
        execlp(command, command, NULL);
    }
    else
    {

```

```
    printf("Parent PID: %d\n", getpid());  
    wait(NULL);  
}  
  
return 0;  
}
```

OUTPUT-

```
rahuuuu@DESKTOP-KDECOD0:~$ nano practical1.c  
rahuuuu@DESKTOP-KDECOD0:~$ gcc practical1.c  
rahuuuu@DESKTOP-KDECOD0:~$ ./a.out  
Enter a linux command: pwd  
Parent PID: 519  
Child PID: 520  
/home/rahuuuu  
rahuuuu@DESKTOP-KDECOD0:~$ |
```

PRACTICAL-2

Q) Develop a C program that uses the system calls `open()`, `read()`, `write()`, and `close()` to copy the contents of one file to another. Explain how control transitions between user space and kernel space during execution.

```
#include <stdio.h>  
#include <unistd.h>  
#include <fcntl.h>
```

```

int main()
{
    int fd1, fd2;
    char message[100];
    int n;

    fd1 = open("sample.txt", O_RDONLY | O_CREAT, 0666);
    fd2 = open("copy.txt", O_WRONLY | O_CREAT, 0666);

    n = read(fd1, message, 100);
    write(fd2, message, n);

    close(fd1);
    close(fd2);

    printf("File copied Successfully\n");

    return 0;
}

```

OUTPUT-

```

rahuuuu@DESKTOP-KDECOD0:~$ ./practical2
File copied Successfully
rahuuuu@DESKTOP-KDECOD0:~$ cat sample.txt
Hello World
rahuuuu@DESKTOP-KDECOD0:~$ cat copy.txt
Hello World
rahuuuu@DESKTOP-KDECOD0:~$ |

```

ii) strace cat sample.txt

Q) Use the strace utility to trace all system calls generated by the command cat sample.txt.

Analyze the sequence of system calls and identify the kernel services involved


```

if(pid < 0)
{
    printf("Fork Failed\n");
}
else if(pid == 0)
{
    printf("Child PID: %d\n", getpid());
    printf("Parent PID: %d\n", getppid());
    sleep(10);
}
else
{
    printf("Parent PID: %d\n", getpid());
    sleep(10);
}

return 0;
}

```

OUTPUT-

```

rahuuuu@DESKTOP-KDECOD0:~$ gcc practical3.c
rahuuuu@DESKTOP-KDECOD0:~$ ./a.out
Parent PID: 1220
Child PID: 1221
Parent PID: 1220
rahuuuu@DESKTOP-KDECOD0:~$ |

```

PRACTICAL-4

Q)Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

```

#include <stdio.h>
#include <unistd.h>

```

```
#include <sys/wait.h>

int main()
{
    int p1, p2, p3;

    p1 = fork();

    if(p1 == 0)
    {
        printf("Child 1 PID: %d\n", getpid());
        return 0;
    }

    p2 = fork();

    if(p2 == 0)
    {
        printf("Child 2 PID: %d\n", getpid());
        return 0;
    }

    p3 = fork();

    if(p3 == 0)
    {
        printf("Child 3 PID: %d\n", getpid());
        return 0;
    }

    waitpid(p2, NULL, 0);

    printf("Parent: Child 2 completed\n");

    return 0;
}
```

OUTPUT-

```
rahuuuu@DESKTOP-KDECODE0:~$ gcc practical4.c
rahuuuu@DESKTOP-KDECODE0:~$ ./a.out
Child 1 PID: 1296
Child 2 PID: 1297
Child 3 PID: 1298
Parent: Child 2 completed
```

(ii) Zombie Process

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

CODE-

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>

int main()
{
    if(fork() == 0)
    {
        printf("Child process\n");
        printf("CP PID is=%d\n", getpid());
        printf("CP execution is done\n");
        exit(0);
    }
    else
    {
        printf("Parent process\n");
        printf("pp pid is = %d\n", getpid());
        sleep(10);
        printf("PP done");
    }

    return 0;
}
```

OUTPUT-

```
rahuuuu@DESKTOP-KDECOD0:~$ nano zombie.c
rahuuuu@DESKTOP-KDECOD0:~$ gcc zombie.c
rahuuuu@DESKTOP-KDECOD0:~$ ./a.out
Parent process
pp pid is = 1374
Child process
CP PID is=1375
CP execution is done
|
```

PRACTICAL-5

Q)Develop a program that executes the equivalent of the shell command: `ls -l | grep ".c"` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int fd[2];
    pid_t p1, p2;

    pipe(fd);

    p1 = fork();

    if(p1 == 0)
    {
        close(fd[0]);

        dup2(fd[1], STDOUT_FILENO);
```

```
        close(fd[1]);

        execlp("ls", "ls", "-l", NULL);
    }

    p2 = fork();

    if(p2 == 0)
    {
        close(fd[1]);

        dup2(fd[0], STDIN_FILENO);

        close(fd[0]);

        execlp("grep", "grep", ".c", NULL);
    }

    close(fd[0]);
    close(fd[1]);

    wait(NULL);
    wait(NULL);

    return 0;
}
```

OUTPUT-

```

rahuuuuu@DESKTOP-KDECODE0:~$ gcc practical5.c
rahuuuuu@DESKTOP-KDECODE0:~$ ./a.out
-rw-r--r-- 1 rahuuuuu rahuuuuu 182 Aug  4 03:56 1.c
-rw-r--r-- 1 rahuuuuu rahuuuuu 257 Aug  4 03:56 2.c
-rw-r--r-- 1 rahuuuuu rahuuuuu 277 Aug  4 03:57 3.c
-rw-r--r-- 1 rahuuuuu rahuuuuu 165 Aug  4 03:58 4.c
-rw-r--r-- 1 rahuuuuu rahuuuuu 218 Aug  4 03:58 5.c
-rw-r--r-- 1 rahuuuuu rahuuuuu 287 Aug  4 03:59 6.c
-rw-r--r-- 1 rahuuuuu rahuuuuu 176 Aug  4 04:00 7.c
-rw-r--r-- 1 rahuuuuu rahuuuuu 12 Sep 15 03:59 copy.txt
-rw-r--r-- 1 rahuuuuu rahuuuuu 161 Aug  4 03:20 fork.c
-rw-r--r-- 1 rahuuuuu rahuuuuu 287 Aug  4 03:33 he.c
-rw-r--r-- 1 rahuuuuu rahuuuuu 176 Aug  4 03:35 hehe.c
-rw-r--r-- 1 rahuuuuu rahuuuuu 165 Aug  4 03:29 hel.c
-rw-r--r-- 1 rahuuuuu rahuuuuu 182 Aug  4 03:23 hello.c
-rw-r--r-- 1 rahuuuuu rahuuuuu 277 Aug  4 03:27 hi.c
-rw-r--r-- 1 rahuuuuu rahuuuuu 666 Aug  4 03:51 last.c
-rw-r--r-- 1 rahuuuuu rahuuuuu 218 Aug  4 03:30 me.c
-rw-r--r-- 1 rahuuuuu rahuuuuu 435 Sep 15 03:36 practical1.c
-rwxr-xr-x 1 rahuuuuu rahuuuuu 16176 Sep 15 03:59 practical2
-rw-r--r-- 1 rahuuuuu rahuuuuu 390 Sep 15 03:57 practical2.c
-rw-r--r-- 1 rahuuuuu rahuuuuu 427 Sep 15 04:00 practical3.c
-rw-r--r-- 1 rahuuuuu rahuuuuu 519 Sep 15 04:06 practical4.c
-rw-r--r-- 1 rahuuuuu rahuuuuu 553 Sep 15 04:08 practical5.c
-rw-r--r-- 1 rahuuuuu rahuuuuu 505 Sep  1 03:35 sjf.c
-rw-r--r-- 1 rahuuuuu rahuuuuu 411 Aug 11 04:02 wl.c
-rw-r--r-- 1 rahuuuuu rahuuuuu 1003 Aug 11 03:52 week1.c
-rw-r--r-- 1 rahuuuuu rahuuuuu 257 Aug  4 03:26 wsp.c
-rw-r--r-- 1 rahuuuuu rahuuuuu 431 Sep 15 04:07 zombie.c
rahuuuuu@DESKTOP-KDECODE0:~$ |

```

(ii). Producer Consumer using Anonymous Pipe
Q) Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

```

#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <fcntl.h>

int main()
{
    int fd[2];

```

```

pid_t pid;

char data[] = "Hello Producer";
char buffer[100];

pipe(fd);

pid = fork();

if(pid == 0)
{
    close(fd[1]);

    read(fd[0], buffer, sizeof(buffer));

    printf("Consumer received: %s\n", buffer);

    close(fd[0]);
}
else
{
    close(fd[0]);

    write(fd[1], data, strlen(data) + 1);

    printf("Producer sent: %s\n", data);

    close(fd[1]);
}

return 0;
}

```

OUTPUT-

```

ahuuuu@DESKTOP-KDECOD0:~$ nano anonypipe.c
ahuuuu@DESKTOP-KDECOD0:~$ gcc anonypipe.c
ahuuuu@DESKTOP-KDECOD0:~$ ./a.out
Producer sent: Hello Producer
Consumer received: Hello Producer
ahuuuu@DESKTOP-KDECOD0:~$ |

```

6(i) FIFO Server -

Q) Create a client-server application using Named Pipes (FIFOs). The client sends messages and the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
#include <string.h>

int main()
{
    int fd;
    char message[100];

    mkfifo("/tmp/myfifo", 0666);

    fd = open("/tmp/myfifo", O_RDONLY);

    read(fd, message, sizeof(message));

    printf("Server received the message: %s\n", message);

    close(fd);

    return 0;
}
```

```

#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <fcntl.h>

int main()
{
    int fd;
    char message1[] = "Hello world";

    fd = open("/tmp/myfifo", O_WRONLY);

    write(fd, message1, strlen(message1) + 1);

    printf("Client sent the message: %s\n", message1);

    close(fd);

    return 0;
}

```

OUTPUT-

```

rahuuuu@DESKTOP-KDECOD0:~$ ./client
Client sent the message: Hello world

```

```

Server received the message: Hello world

```

6(ii) SIGNALS

Q) Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1.

Demonstrate asynchronous event handling and explain the role of signal handlers

```

#include <stdio.h>
#include <signal.h>

void handler(int sig)

```

```

{
    if(sig == SIGINT)
        printf("SIGINT received\n");

    if(sig == SIGTERM)
        printf("SIGTERM received\n");

    if(sig == SIGUSR1)
        printf("SIGUSR1 received\n");
}

int main()
{
    signal(SIGINT, handler);
    signal(SIGTERM, handler);
    signal(SIGUSR1, handler);

    printf("Signal handling program\n");

    raise(SIGINT);
    raise(SIGTERM);
    raise(SIGUSR1);

    return 0;
}

```

OUTPUT-

```

rahuuuu@DESKTOP-KDECOD0:~$ ./a.out
Signal handling program
SIGINT received
SIGTERM received
SIGUSR1 received
rahuuuu@DESKTOP-KDECOD0:~$ |

```

```
rahuuuu@DESKTOP-KDECOD0:~$ ./client
Client sent the message: Hello world
rahuuuu@DESKTOP-KDECOD0:~$ |
```